E23CSEU1873 | LAB11 | LAKSHYA NAYAK

```
!pip install transformers datasets accelerate -q
```

```
import torch, math
from transformers import GPT2LMHeadModel, GPT2Tokenizer, Trainer, TrainingArguments, DataCollatorForLanguageModeling
from datasets import Dataset

tokenizer = GPT2Tokenizer.from_pretrained('distilgpt2')  # lighter than gpt2
model = GPT2LMHeadModel.from_pretrained('distilgpt2')

tokenizer.pad_token = tokenizer.eos_token
model.config.pad_token_id = tokenizer.eos_token_id
```

```
Loading weights: 100%                                    76/76 [00:00<00:00, 268.40it/s, Materializing param=transformer.wte.weight]
GPT2LMHeadModel LOAD REPORT from: distilgpt2
Key                                        | Status     |  |
-------------------------------------------+------------+--+-
transformer.h.{0, 1, 2, 3, 4, 5}.attn.bias | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
```

```
def generate_text(model, tokenizer, prompt, max_length=50):
    device = next(model.parameters()).device   # get model device

    inputs = tokenizer.encode(prompt, return_tensors='pt').to(device)

    model.eval()
    with torch.no_grad():
        outputs = model.generate(
            inputs,
            max_length=max_length,
            temperature=0.8,
            top_k=50,
            do_sample=True,
            pad_token_id=tokenizer.eos_token_id
        )

    return tokenizer.decode(outputs[0], skip_special_tokens=True)
```

```
prompts = ["This product is", "I bought this phone and"]

print("=== BEFORE FINE-TUNING ===")
baseline = {}

for p in prompts:
    baseline[p] = generate_text(model, tokenizer, p)
    print(p, "→", baseline[p])
```

```
=== BEFORE FINE-TUNING ===
This product is → This product is available in just over 2,000 colours.



I bought this phone and → I bought this phone and I can't wait to have it unlocked before I have it unlocked to be able to take it to my house. I can't wait to have it unlocke
```

```
corpus = [
    "this phone has an amazing battery life and the camera quality is outstanding for the price.",
    "i bought this laptop for college and it handles all my assignments perfectly.",
    "the sound quality of these headphones is incredible with deep bass.",
    "this smartwatch tracks my steps accurately.",
    "great wireless earbuds with noise cancellation.",
    "excellent value for money and premium build quality.",
    "highly recommend this product for daily use.",
    "easy to set up and very user friendly.",
]
```

```
dataset = Dataset.from_dict({"text": corpus})

def tokenize(x):
    return tokenizer(x["text"], truncation=True, padding="max_length", max_length=64)

tokenized = dataset.map(tokenize, batched=True, remove_columns=["text"])
split = tokenized.train_test_split(test_size=0.2)
```

```
Map: 100%                                    8/8 [00:00<00:00, 324.05 examples/s]
```

```
dataset = Dataset.from_dict({"text": corpus})

def tokenize(x):
    return tokenizer(x["text"], truncation=True, padding="max_length", max_length=64)
```

```
tokenized = dataset.map(tokenize, batched=True, remove_columns=["text"])
split = tokenized.train_test_split(test_size=0.2)
```

```
Map: 100%                                                    8/8 [00:00<00:00, 393.87 examples/s]
```

```
data_collator = DataCollatorForLanguageModeling(tokenizer=tokenizer, mlm=False)

training_args = TrainingArguments(
    output_dir="./model",
    num_train_epochs=5,  # keep low for fast run
    per_device_train_batch_size=2,
    logging_steps=5,
    save_strategy="no"
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=split["train"],
    eval_dataset=split["test"],
    data_collator=data_collator
)

trainer.train()
```

[15/15 00:00, Epoch 5/5]

Step	Training Loss
5	4.027571
10	2.869329
15	2.546700

```
TrainOutput(global_step=15, training_loss=3.147866948445638, metrics={'train_runtime': 0.7735, 'train_samples_per_second': 38.787, 'train_steps_per_second': 19.393,
'total_flos': 489931407360.0, 'train_loss': 3.147866948445638, 'epoch': 5.0})
```

```
print("\n=== AFTER FINE-TUNING ===")

for p in prompts:
    output = generate_text(model, tokenizer, p)
    print(p, "→", output)
```

```
=== AFTER FINE-TUNING ===
This product is → This product is for those of you who are in the consumer of audio and gaming for hours at a time.

I bought this phone and → I bought this phone and had no problems. The included lock key and it is very compact.
```

```
tokenizer2 = GPT2Tokenizer.from_pretrained('distilgpt2')
model2 = GPT2LMHeadModel.from_pretrained('distilgpt2')

tokenizer2.pad_token = tokenizer2.eos_token
model2.config.pad_token_id = tokenizer2.eos_token_id
```

```
Loading weights: 100%                                    76/76 [00:00<00:00, 417.95it/s, Materializing param=transformer.wte.weight]
GPT2LMHeadModel LOAD REPORT from: distilgpt2
Key                                      | Status     |  |
-----------------------------------------+------------+--+-
transformer.h.{0, 1, 2, 3, 4, 5}.attn.bias | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
```

```
recipes = [
    "to make butter chicken marinate chicken with spices and yogurt.",
    "heat butter and cook onions until golden brown.",
    "add tomato puree and cook for 10 minutes.",
    "add chicken and cook until tender.",
    "finish with cream and serve hot.",

    "boil pasta until al dente.",
    "fry pancetta until crispy.",
```

```
    "mix eggs and cheese.",
    "combine pasta with mixture to form sauce.",

    "heat oil and add vegetables.",
    "add soy sauce and stir fry.",
    "serve hot with rice."
]
```

```
dataset2 = Dataset.from_dict({"text": recipes})

tokenized2 = dataset2.map(
    lambda x: tokenizer2(x["text"], truncation=True, padding="max_length", max_length=64),
    batched=True,
    remove_columns=["text"]
)

split2 = tokenized2.train_test_split(test_size=0.2)

trainer2 = Trainer(
    model=model2,
    args=training_args,
    train_dataset=split2["train"],
    eval_dataset=split2["test"],
    data_collator=DataCollatorForLanguageModeling(tokenizer=tokenizer2, mlm=False)
)

trainer2.train()
```

Map: 100% 12/12 [00:00<00:00, 622.45 examples/s]

[25/25 00:01, Epoch 5/5]

Step	Training Loss
5	4.444715
10	3.251677
15	2.750665
20	2.513272
25	2.372787

```
TrainOutput(global_step=25, training_loss=3.0666234207153322, metrics={'train_runtime': 1.1972, 'train_samples_per_second': 37.589, 'train_steps_per_second': 20.883,
'total_flos': 734897111040.0, 'train_loss': 3.0666234207153322, 'epoch': 5.0})
```

```
prompts2 = ["To make butter chicken", "For pasta"]

print("\n=== RECIPE OUTPUT ===")

for p in prompts2:
    print(p, "→", generate_text(model2, tokenizer2, p))
```

```
=== RECIPE OUTPUT ===
To make butter chicken → To make butter chicken and cook till golden brown. Add flour and cook until golden brown. Add chicken and cook until tender. Add vegetables and cook u
For pasta → For pasta and cook until golden brown. Add pasta and cook until golden brown. Add pasta and cook until golden brown. Add pasta and cook until golden brown. Add pas
```